



南開大學
Nankai University

计算机学院
自然语言处理实验报告

文本分类实现

姓 名：余俊辉

学 号：2313486

专 业：计算机科学与技术

2026 年 10 月 9 日

目录

1	实验目的	2
2	实验环境与数据集	2
2.1	数据集介绍	2
2.2	数据划分与预处理	2
2.3	实验环境	3
3	评价指标	4
4	Task 1: 词袋模型	4
4.1	方法原理	4
4.2	实现细节	5
4.3	实验结果	5
5	Task 2: 词向量表示	6
5.1	方法原理	6
5.2	三组实验设置与实现	6
5.3	实验结果	7
6	Task 3: BERT 微调	8
6.1	方法原理	8
6.2	实现细节	8
6.3	训练过程与结果	9
7	实验结果与分析	10
7.1	总体结果	10
7.2	词袋为什么反超平均词向量	10
7.3	词向量来源: 规模、覆盖与领域	11
7.4	BERT 的表现与截断的影响	12
7.5	类别不均衡与指标选择	12
8	实验总结	13
A	代码结构与运行说明	13
A.1	代码结构	13
A.2	数据与模型准备	13
A.3	运行方式	14

1 实验目的

文本分类是自然语言处理中最基础也最典型的任务之一，它要求把一段文本映射到预先给定的类别。同一份文本可以用完全不同的方式表示成计算机能够处理的向量，而表示方式的好坏往往直接决定了分类器的上限。本次实验围绕 New York Times (NYT) 新闻三分类任务，依次实现并比较三类代表性的文本表示方法：基于词项统计的词袋模型 (Bag-of-Words)、基于稠密词向量的平均文档表示 (Word2Vec / GloVe)，以及对预训练语言模型 BERT 进行微调。三种方法共用同一份数据划分和同一套评价指标，以便把差异归因于“表示方法”本身，而不是数据或评估口径的不同。

通过本次实验，希望达到以下目的：

- (1) 理解词袋、词向量与预训练语言模型这三类文本表示的基本思想与各自的适用场景；
- (2) 走通“数据读取 → 划分与预处理 → 特征表示 → 训练分类器 → 测试评估”这一完整的文本分类实验流程；
- (3) 掌握 Logistic Regression 在高维文本特征上的使用方法；
- (4) 掌握预训练 GloVe 的加载，以及使用 gensim 在外部语料上训练 Word2Vec 的方法；
- (5) 熟悉使用 Hugging Face Transformers 对 BERT 进行文本分类微调的流程；
- (6) 能够正确实现并解释 Accuracy 与 Macro-F1 两项指标，并据此对不同方法进行比较和分析。

2 实验环境与数据集

2.1 数据集介绍

实验使用两份数据，均为英文新闻文本：

- **NYT 数据集**：主数据集，文件 `nyt.csv`，包含 `text`、`label` 两列，共 11,519 条新闻，类别为 `sports`、`business`、`politics` 三类。三个任务的模型都在该数据集上训练与评测。
- **AG News 数据集**：文件 `ag.csv`，仅含 `text` 一列，共 90,000 条新闻。本实验中它不参与分类训练，只作为额外语料，用于训练一组 Word2Vec 词向量。

NYT 的类别分布并不均衡，如表 1 所示，`sports` 一类就占了约四分之三，另外两类各占约八分之一。这一特点会直接影响后面指标的选择（详见第 3 节与第 7 节的讨论）。

表 1: NYT 数据集的类别分布

类别	business	politics	sports	合计
文档数量	1,429	1,451	8,639	11,519
占比	12.4%	12.6%	75.0%	100%

2.2 数据划分与预处理

按照作业要求，先对 NYT 做随机打乱，再按 80%/10%/10% 划分为训练集、验证集和测试集。为了让结果可以复现，随机种子固定为 `RANDOM_SEED=42`；同时采用分层抽样 (`stratify`)，使三个集合中的类别比例与整体保持一致。具体做法是两次调用 `train_test_split`：先切出 10% 作为测试集，再从

剩余 90% 中切出 1/9（即总量的 10%）作为验证集，剩下的 80% 作为训练集，最终得到训练集 9,215 条、验证集 1,152 条、测试集 1,152 条，各集合的类别计数见表 2，对应分布见图 2.1。

表 2: 固定随机种子（42）下的分层数据划分

集合	business	politics	sports	合计
训练集 Train (80%)	1,143	1,161	6,911	9,215
验证集 Validation (10%)	143	145	864	1,152
测试集 Test (10%)	143	145	864	1,152

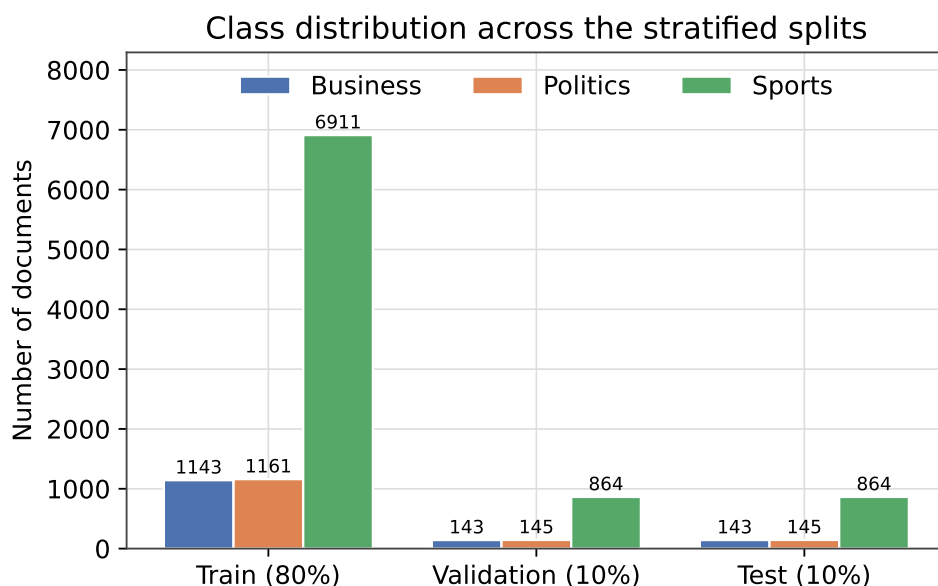


图 2.1: 分层划分后各集合的类别分布，三个集合的类别比例与整体基本一致

需要强调的是，三个任务始终复用同一份划分，最终性能统一在测试集（NYT Test Set）上报告，保证六种实验设置之间的结果可以直接比较。

预处理方面，Task 1 和 Task 2 使用 NLTK 的 `word_tokenize` 进行英文分词，并统一转为小写（预训练 GloVe 词表本身为全小写，统一大小写可以减少未登录词，也保证各方法预处理口径一致）。Task 3 不使用这一分词器，而是使用 BERT 自带的 WordPiece 子词分词器。为了避免数据泄漏，词袋的词表、Word2Vec 词向量都只在**训练集**上学习，验证集和测试集只做相同的变换；在 NYT 上自训的 Word2Vec 也只使用训练集文本，而不接触验证集与测试集。

2.3 实验环境

实验在一台配备 NVIDIA RTX 4060 Laptop（8 GB 显存）的笔记本上完成，主要软件版本见表 3。Task 1、Task 2 在 CPU 上即可快速完成；Task 3 在 GPU 上以半精度（fp16）微调，3 个 epoch 完整训练约 1 分 51 秒。

表 3: 实验环境与主要依赖版本

项目	版本 / 说明
操作系统	Windows (x64)
Python	3.11
PyTorch	2.14.1 (CUDA 12.6, GPU)
Transformers	5.19.0
Accelerate	1.15.0
scikit-learn	1.9.1
gensim	4.4.0
NLTK	3.10.3
pandas / numpy	3.0.6 / 2.4.6
预训练模型	google-bert/bert-base-uncased、glove.6B.100d

3 评价指标

准确率 (Accuracy) 衡量分类正确的样本占全部样本的比例。设测试集共 N 条样本，第 i 条的真实标签为 y_i 、预测标签为 $\hat{y}_i$ ，则

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i). \quad (1)$$

宏平均 F1 (Macro-F1) 先逐个类别计算 F1，再对所有类别做无加权平均。对类别 c ，记真正例、假正例、假负例分别为 TP_c 、 FP_c 、 FN_c ，则

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}, \quad (2)$$

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c, \quad (3)$$

其中 C 为类别数。若某个类别既没有真实样本也没有预测样本，约定其 F1 为 0（与 scikit-learn 的 `zero_division=0` 口径一致）。

在类别均衡时 Accuracy 直观且足够，但在本数据集这种大类（sports）占绝大多数的情况下，即使模型对小类判得很差，Accuracy 仍然会显得很高。Macro-F1 对每个类别等权平均，不会被大类“带飞”，因此更能反映模型在 business、politics 两个小类上的真实能力。本实验同时报告两项指标，并以 Macro-F1 作为主要比较依据。

两项指标均在 `eval_utils.py` 中手写实现。为了确认实现正确，本文将其结果与 scikit-learn 的 `accuracy_score`、`f1_score(average='macro')` 逐位对拍，结果完全一致，包括某一类零预测的边界情形。

4 Task 1: 词袋模型

4.1 方法原理

词袋模型把文档看成一个无序的“词袋子”，忽略词序、句法和语法，只关心词表中的词在文档中是否出现、出现多少次。设训练语料构建出的词表大小为 $|V|$ ，则每篇文档被表示为一个 $|V|$ 维的稀疏

向量。

Binary Bag of Words 只记录词是否出现，向量第 i 维为

$$x_i = \begin{cases} 1, & \text{若词表中第 } i \text{ 个词出现在该文档中,} \\ 0, & \text{否则.} \end{cases} \quad (4)$$

Word Frequency (词频) 则记录出现次数，向量第 i 维取该词在文档中的计数 c_i ：

$$\mathbf{x} = (c_1, c_2, \dots, c_{|V|}). \quad (5)$$

两者的区别在于：二值表示认为一个词出现一次和出现多次没有差别，词频表示则保留了这种强弱信息。

分类器统一使用 Logistic Regression。对 K 分类问题，它对类别 k 建模一个线性打分 $\mathbf{w}_k^\top \mathbf{x} + b_k$ ，再经 softmax 归一化为概率：

$$p(y = k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}, \quad (6)$$

训练时最大化带 L_2 正则的对数似然（等价于最小化交叉熵损失加正则项）。词袋特征维度高、极度稀疏，但不同类别往往由少量高度指示性的词线性决定，因此线性的 Logistic Regression 在这种特征上通常已经有很好的表现。

4.2 实现细节

直接使用 scikit-learn 的 `CountVectorizer` 构建词袋，`analyzer` 指定为项目统一的分词函数（其内部已完成小写化），通过 `binary` 开关在二值词袋与词频之间切换。关键在于词表只在训练集上 `fit` 一次，验证集、测试集仅做 `transform`，杜绝测试集信息泄漏。本数据集构建出的词表规模为 127,018 维。核心代码如下：

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.linear_model import LogisticRegression
3
4 # fit on TRAIN only; binary=True -> Binary BoW, binary=False -> word frequency
5 vectorizer = CountVectorizer(analyzer=tokenize, binary=binary)
6 X_train = vectorizer.fit_transform(train_texts)
7 X_val = vectorizer.transform(val_texts)
8 X_test = vectorizer.transform(test_texts)
9
10 clf = LogisticRegression(max_iter=1000)
11 clf.fit(X_train, y_train)
12 y_pred = clf.predict(X_test)
```

4.3 实验结果

两种词袋表示在测试集上的结果见表 4。两者的 Accuracy 相同 (0.9826)，但词频表示的 Macro-F1 更高 (0.9621 对 0.9576)，说明“出现多少次”这一信息主要改善了小类别的判别，这一点在 Accuracy 上看不出来，却被 Macro-F1 捕捉到了。

表 4: Task 1 在 NYT 测试集上的结果

表示方法	Accuracy	Macro-F1
Binary Bag of Words + LR	0.9826	0.9576
Word Frequency + LR	0.9826	0.9621

```
(nlp-hw1) PS D:\aaadep\grade_4\自然语言\NKU-NLP\hw1> python .\task1_bow.py
词袋矩阵形状 train=(9215, 127018) test=(1152, 127018)
[Task1-Binary-BoW] Accuracy=0.9826 Macro-F1=0.9576
词袋矩阵形状 train=(9215, 127018) test=(1152, 127018)
[Task1-Word-Frequency] Accuracy=0.9826 Macro-F1=0.9621

===== Task 1 =====
{'experiment': 'Task1-Binary-BoW', 'accuracy': 0.9826, 'macro_f1': 0.9576}
{'experiment': 'Task1-Word-Frequency', 'accuracy': 0.9826, 'macro_f1': 0.9621}
```

图 4.2: Task 1 词袋模型运行结果：终端打印两种词袋的矩阵形状与 Accuracy / Macro-F1

5 Task 2: 词向量表示

5.1 方法原理

词袋向量维度高、稀疏，且任意两个维度之间相互独立，无法表达“近义词”这类语义关系。词向量（word embedding）则把每个词映射为一个低维、稠密的实值向量，向量之间的距离和方向能够编码语义相似性，其理论基础是分布式假设：出现在相似上下文中的词往往具有相似的含义。

Word2Vec 是最具代表性的词向量学习方法之一，包含 CBOW 和 Skip-gram 两种结构。本实验使用 gensim 默认的 CBOW：在一个上下文窗口内，用周围词的向量预测中心词，训练完成后，网络的权重视为每个词的稠密向量。**GloVe** 则走另一条路线，它先基于全局语料统计词与词的共现次数，再通过矩阵分解让词向量的点积拟合共现的对数加权值，从而同时利用全局统计信息和局部上下文信息。

词向量是词级别的表示，要得到文档级别的特征，需要做一次聚合。按照作业要求，对一篇包含 n 个有效词的文档，取其词向量的平均作为文档向量：

$$\mathbf{e}(d) = \frac{1}{n} \sum_{i=1}^n \mathbf{e}(w_i). \quad (7)$$

实现时只对词表中存在的词（有效词）求平均，未登录词（OOV）直接跳过；若整篇文档没有任何有效词，则返回全零向量以避免除零。最后仍然用 Logistic Regression 在 100 维文档向量上分类。

5.2 三组实验设置与实现

三组实验使用完全相同的文档聚合与分类流程，区别只在于 100 维词向量的来源：

- (1) **预训练 GloVe**: 加载 glove.6B.100d，该模型在约 60 亿 token 的通用语料上训练，词表约 40 万词。文件每行是“一个词 + 100 个数值”，逐行解析为“词 $\rightarrow$ 向量”的字典即可。
- (2) **在 AG News 上自训 Word2Vec**: 对 90,000 条 AG News 文本分词后训练 CBOW 模型。

(3) 在 NYT 上自训 Word2Vec: 仅用 NYT 训练集文本训练 CBOW 模型, 以考察同领域语料的效果 (同样不使用验证、测试集, 防止泄漏)。

两组自训 Word2Vec 的超参数保持一致: 向量维度 100、上下文窗口 5、最低词频 `min_count=5`、训练 5 个 epoch、固定随机种子 42。为了和 GloVe 统一接口, 训练好的 gensim 模型也导出为“词 → 向量”的字典, 再走相同的文档平均与分类流程。关键实现如下:

```
1 from gensim.models import Word2Vec
2
3 # CBOW, 100-dim, window 5, drop words appearing fewer than 5 times
4 model = Word2Vec(sentences=sentences, vector_size=100, window=5,
5                 min_count=5, workers=4, epochs=5, seed=42)
6
7
8 import numpy as np
9
10 # average the embeddings of in-vocabulary words; OOV words are skipped
11 def doc_vector(tokens, embeddings):
12     vectors = [embeddings[w] for w in tokens if w in embeddings]
13     if len(vectors) == 0:
14         return np.zeros(100, dtype=np.float32) # all-OOV document
15     return np.mean(np.stack(vectors), axis=0)
```

5.3 实验结果

三组词向量在测试集上的结果见表 5。预训练 GloVe 最好, 在规模更大的 AG News 上自训的 Word2Vec 次之, 仅用 NYT 训练集自训的略低; 三者整体都略低于 Task 1 的词袋模型。两组自训词向量之间的差距很小 (Accuracy 仅相差 0.08 个百分点), 具体原因将在第 7 节统一分析。

表 5: Task 2 在 NYT 测试集上的结果

词向量来源	Accuracy	Macro-F1
预训练 GloVe (glove.6B.100d)	0.9774	0.9468
Word2Vec, 自训于 AG News (9 万篇)	0.9748	0.9406
Word2Vec, 自训于 NYT 训练集 (9 千篇)	0.9740	0.9391

```
(nlp-hw1) PS D:\aaadep\grade_4\自然语言\NKU-NLP\hw1> python .\task2_word2vec.py
Task2-GloVe 特征矩阵 train=(9215, 100) test=(1152, 100)
[Task2-GloVe] Accuracy=0.9774 Macro-F1=0.9468
Task2-W2V-AGNews 特征矩阵 train=(9215, 100) test=(1152, 100)
[Task2-W2V-AGNews] Accuracy=0.9748 Macro-F1=0.9406
Task2-W2V-NYT 特征矩阵 train=(9215, 100) test=(1152, 100)
[Task2-W2V-NYT] Accuracy=0.9740 Macro-F1=0.9391

===== Task 2 =====
{'experiment': 'Task2-GloVe', 'accuracy': 0.9774, 'macro_f1': 0.9468}
{'experiment': 'Task2-W2V-AGNews', 'accuracy': 0.9748, 'macro_f1': 0.9406}
{'experiment': 'Task2-W2V-NYT', 'accuracy': 0.974, 'macro_f1': 0.9391}
```

图 5.3: Task 2 词向量模型运行结果: 终端依次打印 GloVe、AG-W2V、NYT-W2V 的特征矩阵形状与指标

6 Task 3: BERT 微调

6.1 方法原理

BERT (Bidirectional Encoder Representations from Transformers) 建立在多层 Transformer 编码器之上，核心是多头自注意力机制。与单向语言模型不同，BERT 通过掩码语言模型 (MLM) 和下一句预测 (NSP) 两个预训练任务，在大规模通用语料上学习到了真正双向的上下文表示，同一个词在不同语境下会得到不同的向量，这从根本上区别于 Task 2 中“一个词只有一个固定向量”的静态词向量。

用于分类时，BERT 采用 WordPiece 子词分词，在序列首尾分别加上特殊标记 [CLS]_ 与 [SEP]_，把 [CLS]_ 对应的最终隐状态接入一个线性分类头，输出各类别 logits。预训练已经提供了很好的通用语言表示，因此下游任务通常只需要用标注数据对全部参数做少量轮次的 **微调 (fine-tuning)** 即可。

6.2 实现细节

使用 Hugging Face Transformers 实现。代码优先加载随项目下载到本地的 HW-1/bert-base-uncased，目录缺失时再回退到在线仓库。分词时按作业要求设置 `max_length=64`，超长截断、不足则填充到等长：

```
1 from transformers import AutoTokenizer, AutoModelForSequenceClassification
2
3 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
4 encodings = tokenizer(list(texts),
5                       truncation=True,
6                       padding="max_length",
7                       max_length=64)
8 model = AutoModelForSequenceClassification.from_pretrained(
9     MODEL_NAME, num_labels=3, id2label=id2label, label2id=label2id)
```

训练通过 Trainer 完成，训练 3 个 epoch，训练批大小 16、评估批大小 32，每个 epoch 在验证集上评估一次，GPU 可用时自动开启 fp16；评估回调复用本项目自己实现的 Accuracy 与 Macro-F1 (对 logits 取 argmax 后计算)。

```
1 training_args = TrainingArguments(
2     output_dir="./results/bert",
3     num_train_epochs=3,
4     per_device_train_batch_size=16,
5     per_device_eval_batch_size=32,
6     eval_strategy="epoch",
7     save_strategy="no",
8     logging_steps=100,
9     fp16=torch.cuda.is_available(),
10    report_to=[])
11 trainer = Trainer(model=model, args=training_args,
12                  train_dataset=train_ds, eval_dataset=val_ds,
13                  processing_class=tokenizer, compute_metrics=compute_metrics)
14 trainer.train()
```

需要说明的是，加载预训练权重时终端会提示 `cls.predictions`、`cls.seq_relationship` 等预训练头为“多余权重 (UNEXPECTED)”，而 `classifier.weight/bias` 为“缺失权重 (MISSING)”。这是正常现象：前者是预训练阶段的 MLM/NSP 头，分类任务用不到；后者是下游分类头，本就没有

预训练权重、需要随机初始化后再微调。

6.3 训练过程与结果

训练共 1,728 个优化步，在 GPU 上以半精度训练耗时约 111 秒（1 分 51 秒）。验证集指标随 epoch 的变化见表 6 与图 6.4：第 2 个 epoch 达到峰值（Accuracy 0.9809、Macro-F1 0.9599），第 3 个 epoch 略有回落；与此同时验证损失从第 1 个 epoch 的 0.0728 一路上升到第 3 个 epoch 的 0.0849，指标先升后降、损失却持续走高，是比较典型的轻微过拟合迹象。最终在测试集上得到 Accuracy 0.9774、Macro-F1 0.9517。

表 6: BERT 各 epoch 验证集指标与最终测试集结果

评估阶段	验证损失	Accuracy	Macro-F1
验证集 Epoch 1	0.0728	0.9783	0.9527
验证集 Epoch 2	0.0823	0.9809	0.9599
验证集 Epoch 3	0.0849	0.9800	0.9545
测试集（最终结果）	—	0.9774	0.9517

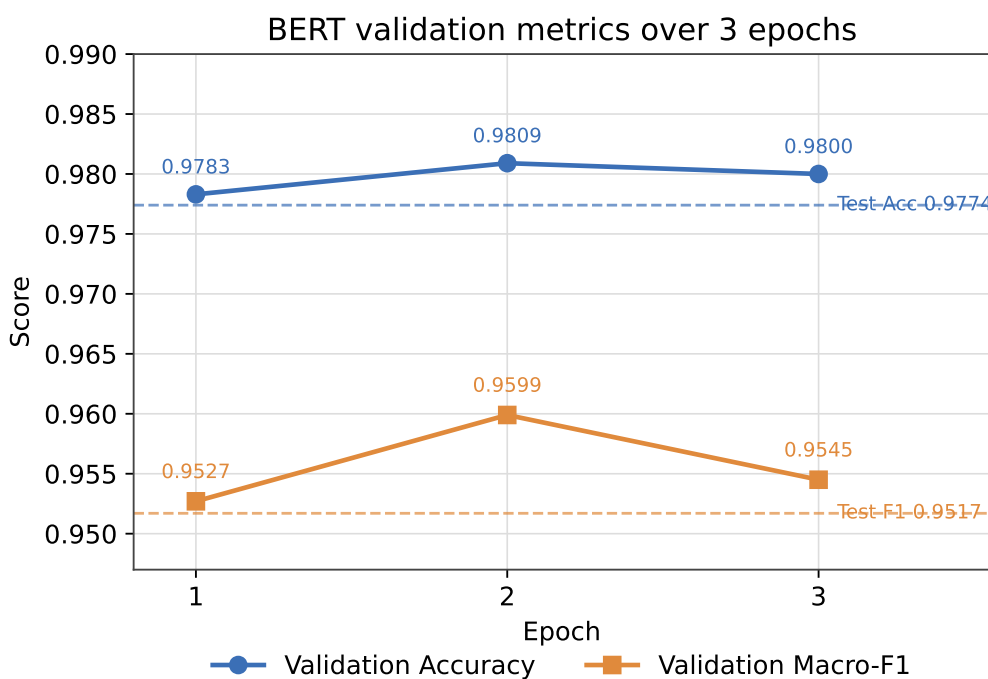


图 6.4: BERT 三个 epoch 的验证集指标，虚线为最终测试集结果

```
{'loss': '0.2522', 'grad_norm': '3.348', 'learning_rate': '4.714e-05', 'epoch': '0.1736'}
{'loss': '0.1699', 'grad_norm': '9.92', 'learning_rate': '4.424e-05', 'epoch': '0.3472'}
{'loss': '0.1115', 'grad_norm': '9.616', 'learning_rate': '4.135e-05', 'epoch': '0.5208'}
{'loss': '0.1237', 'grad_norm': '1.985', 'learning_rate': '3.845e-05', 'epoch': '0.6944'}
{'loss': '0.08866', 'grad_norm': '0.2282', 'learning_rate': '3.556e-05', 'epoch': '0.8681'}
{'eval_loss': '0.07281', 'eval_accuracy': '0.9783', 'eval_macro_f1': '0.9527', 'eval_runtime': '0.8698', 'eval_samples_per_second': '1324', 'eval_steps_per_second': '41.39', 'epoch': '1'}
{'loss': '0.0897', 'grad_norm': '0.02028', 'learning_rate': '3.267e-05', 'epoch': '1.042'}
{'loss': '0.04481', 'grad_norm': '0.006684', 'learning_rate': '2.977e-05', 'epoch': '1.215'}
{'loss': '0.0429', 'grad_norm': '0.06178', 'learning_rate': '2.688e-05', 'epoch': '1.389'}
{'loss': '0.06021', 'grad_norm': '21.39', 'learning_rate': '2.399e-05', 'epoch': '1.562'}
{'loss': '0.0494', 'grad_norm': '13.41', 'learning_rate': '2.109e-05', 'epoch': '1.736'}
{'loss': '0.04097', 'grad_norm': '0.3816', 'learning_rate': '1.82e-05', 'epoch': '1.91'}
{'eval_loss': '0.08229', 'eval_accuracy': '0.9809', 'eval_macro_f1': '0.9599', 'eval_runtime': '0.8494', 'eval_samples_per_second': '1356', 'eval_steps_per_second': '42.38', 'epoch': '2'}
{'loss': '0.03815', 'grad_norm': '0.03002', 'learning_rate': '1.531e-05', 'epoch': '2.083'}
{'loss': '0.01575', 'grad_norm': '0.006961', 'learning_rate': '1.241e-05', 'epoch': '2.257'}
{'loss': '0.008528', 'grad_norm': '0.004815', 'learning_rate': '9.52e-06', 'epoch': '2.431'}
{'loss': '0.009037', 'grad_norm': '0.9865', 'learning_rate': '6.626e-06', 'epoch': '2.604'}
{'loss': '0.00902', 'grad_norm': '0.00224', 'learning_rate': '3.733e-06', 'epoch': '2.778'}
{'loss': '0.01919', 'grad_norm': '0.003495', 'learning_rate': '8.391e-07', 'epoch': '2.951'}
{'eval_loss': '0.08492', 'eval_accuracy': '0.98', 'eval_macro_f1': '0.9545', 'eval_runtime': '0.841', 'eval_samples_per_second': '1370', 'eval_steps_per_second': '42.81', 'epoch': '3'}
{'train_runtime': '111.2', 'train_samples_per_second': '248.6', 'train_steps_per_second': '15.54', 'train_loss': '0.06804', 'epoch': '3'}
100% | 1728/1728 [01:51<00:00, 15.54it/s]
100% | 36/36 [00:00<00:00, 44.73it/s]
[Task3-BERT] Accuracy=0.9774 Macro-F1=0.9517

===== Task 3 =====
{'experiment': 'Task3-BERT', 'accuracy': 0.9774, 'macro_f1': 0.9517}
```

图 6.5: Task 3 BERT 微调训练与测试结果：终端打印各 epoch 的训练/验证损失与指标，以及最终测试集 Accuracy / Macro-F1

7 实验结果与分析

7.1 总体结果

六种实验设置在同一份测试集（1,152 条）上的结果汇总于表 7，对应的可视化对比如图 7.6。

表 7: 六种实验设置在 NYT 测试集上的总体结果

任务	方法	Accuracy	Macro-F1
Task 1	Binary BoW + LR	0.9826	0.9576
Task 1	Word Frequency + LR	0.9826	0.9621
Task 2	GloVe 平均向量 + LR	0.9774	0.9468
Task 2	AG News 自训 Word2Vec + LR	0.9748	0.9406
Task 2	NYT 自训 Word2Vec + LR	0.9740	0.9391
Task 3	BERT 微调	0.9774	0.9517

7.2 词袋为什么反超平均词向量

从分数上看，最简单的词袋反而表现最好，Task 2 的平均词向量在 Accuracy 上落后约 0.5~0.9 个百分点，在 Macro-F1 上落后约 1.6~2.3 个百分点。这并不矛盾，原因在于平均这一聚合方式损失了两类对主题分类至关重要的信息。其一，**词频信息被抹平**：一个强指示词（如球队名、公司名、政治人物名）出现一次和反复出现，在平均向量里几乎没有区别，而这正是词频词袋显式保留的。其二，**不同词的贡献被相互抵消**：把成百上千个 100 维向量做算术平均后，文档被压缩成一个单一向量，区分性词汇的信号被大量中性词稀释，词序信息也完全丢失。

相比之下，词袋保留了 127,018 维的细粒度词项特征。虽然维度很高，但特征稀疏、类别近似线

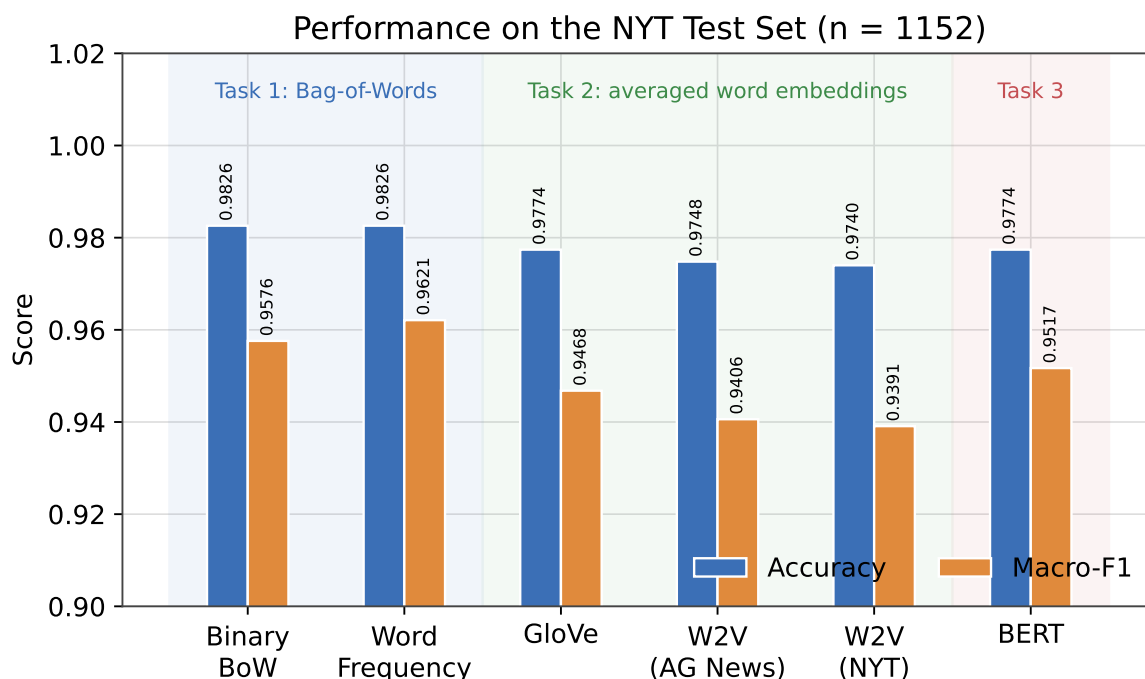


图 7.6: 六种实验设置在 NYT 测试集上的 Accuracy 与 Macro-F1 对比

性可分，配合带 L_2 正则的 Logistic Regression，恰好能稳定地从中学到每个指示词的权重。换句话说，在“主题分类 + 指示词明确”这类任务上，高维稀疏的词袋并不是劣势，反而把最有用的信息完整地留给了线性分类器。

7.3 词向量来源：规模、覆盖与领域

本次运行中 Task 2 内部的排序是 GloVe > AG News 自训 > NYT 自训，可以从语料规模、词覆盖率和领域相关性三个角度解释。

- **GloVe 明显最优：**它在约 60 亿 token 的通用语料上训练，词表约 40 万、覆盖率高，很多在 NYT 中出现次数很少、会被 `min_count=5` 过滤掉的词，在 GloVe 中仍有向量，因此文档平均时有效词更多、表示更稳定，Accuracy 与 Macro-F1 都明显领先两组自训词向量。
- **AG News 自训略优于 NYT 自训：**AG News 有 9 万篇，约为 NYT 训练集（9 千篇）的 10 倍，更大的语料让更多词满足 `min_count=5`，词向量也训练得更充分，本次运行中以很小的优势（Accuracy 高 0.08、Macro-F1 高 0.15 个百分点）领先。
- 需要说明的是，两组自训词向量的这一差距非常小，落在神经网络训练随机性可能造成的数值波动范围之内，不宜据此断言“大规模跨领域语料一定优于同领域小语料”；本实验能稳妥得出的结论是：当语料规模相差约一个数量级时，规模与覆盖率的收益至少不弱于、甚至略超过领域匹配带来的收益。而无论哪一组自训词向量，受语料总量所限，覆盖率都远不及在 60 亿 token 上预训练的 GloVe。

7.4 BERT 的表现与截断的影响

BERT 的 Macro-F1 为 0.9517，高于 Task 2 中最好的平均词向量（GloVe 的 0.9468），Accuracy 则与 GloVe 持平（均为 0.9774），说明上下文相关表示加微调在小类判别上确实强于“静态向量 + 平均”。但它的两项指标都没有超过使用全文的词袋（Accuracy 0.9826、Macro-F1 0.9621）。结合实验设置，本文认为主要有两点原因。

第一，任务本身的上限。 NYT 三分类是典型的主题分类，类别由若干高度指示性的词汇即可基本确定，词袋加线性分类器在这套数据上已经接近性能天花板，留给 BERT 的提升空间本就有限。

第二，也是更关键的一点，输入长度被截断。 作业要求 `max_length=64`，而 NYT 是完整的新闻文章。本文对全部文档按 NLTK 分词统计了长度，如图 7.7：文档平均约 730 个词、中位数 768 词，最短 38 词、最长 5,977 词，**99.88% 的文档都超过了 64 个词**。考虑到 BERT 的 WordPiece 还会把一个词切成更细的子词，实际能容纳的完整单词更少。也就是说，BERT 在三个任务里唯一只看到了文章开头的一小段，正文绝大部分内容都被丢弃；而 Task 1、Task 2 的特征是基于全文构建的。在这种信息不对等的情况下，BERT 仍能在 Macro-F1 上超过全部平均词向量、在 Accuracy 上与最优的 GloVe 持平，已经体现出预训练表示的能力。此外，验证集指标在第 2 个 epoch 见顶后回落、验证损失却从 0.0728 单调上升到 0.0849，说明在 9,215 条数据上对 1.1 亿参数的模型继续训练开始出现轻微过拟合，3 个 epoch 已经足够。可以合理推测，若放宽最大长度（或对长文做分段、滑动窗口等截断策略改进），让 BERT 看到更完整的文章，其性能应当能够超过词袋；这一点可作为后续改进方向。

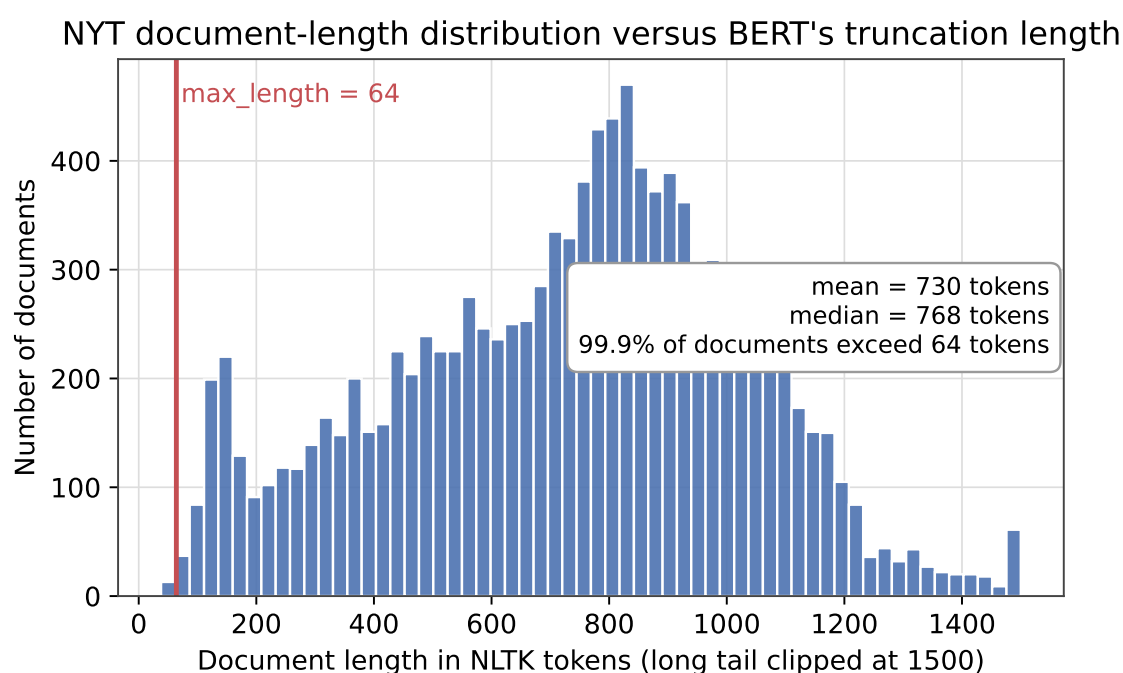


图 7.7: NYT 文档长度分布与 BERT 截断长度（64）的对比，绝大多数文档远超该长度

7.5 类别不均衡与指标选择

由于 sports 占约 75%，一个始终预测 sports 的平凡分类器也能拿到约 75% 的 Accuracy，这说明 Accuracy 在本任务上会被大类明显抬高。Macro-F1 对三个类别等权平均，business、politics 上的改善不会被 sports 掩盖。一个直接的例子是 Task 1：二值词袋与词频词袋的 Accuracy 完全相同

(0.9826)，但 Macro-F1 相差 0.45 个百分点，差异正来自两个小类。这也印证了在不均衡数据集上应当以 Macro-F1 为主、Accuracy 为辅来评价模型。

此外，本文通过固定随机种子保证了数据划分与 Word2Vec 训练的可复现性；不同硬件、不同库版本可能带来合理范围内的微小数值波动，评价指标的手写实现也已与 scikit-learn 对拍一致，因此上述横向比较是可靠的。

8 实验总结

本次实验在同一份 NYT 数据、同一套划分和指标下，完整实现并比较了词袋、平均词向量和 BERT 三类文本表示。结果显示，文本分类的效果并不与模型复杂度简单地成正比：在主题明确、指示词清晰的任务上，朴素的词频词袋配合 Logistic Regression 取得了最高的 Macro-F1 (0.9621)，两种词袋也并列取得了最高的 Accuracy (0.9826)；预训练 GloVe 凭借大规模语料和高覆盖率，在平均词向量方法中明显最好，两组自训词向量中，语料规模大约 10 倍的 AG News 略优于仅用 9 千篇 NYT 训练集的结果，说明在规模差距悬殊时覆盖率的收益并不弱于领域匹配；BERT 的 Macro-F1 高于全部平均词向量、Accuracy 与 GloVe 持平，但受限于 `max_length=64` 对长文的截断，未能超过使用全文的词袋。

通过这组对比可以体会到，选择文本表示时不能只看模型是否“先进”，还要看它与任务特点、输入长度、数据规模和预处理方式是否匹配。预训练语言模型能力很强，但它的实际表现同样会受到输入截断、训练轮数等工程因素的制约；而看似简单的传统方法，在合适的条件下依然是很强的基线。后续可以从放宽输入长度、改进长文聚合方式、按类别加权缓解不均衡等方向继续提升。

附录 A 代码结构与运行说明

A.1 代码结构

完整代码已托管在 GitHub: <https://github.com/Bianssss/NKU-NLP/tree/main/hw1>，主要文件见表 8。

表 8: 项目文件说明

文件	说明
<code>data_utils.py</code>	数据读取、固定种子的分层划分、NLTK 分词、标签映射
<code>eval_utils.py</code>	手写 Accuracy 与 Macro-F1
<code>task1_bow.py</code>	Task 1: 二值/词频词袋 + Logistic Regression
<code>task2_word2vec.py</code>	Task 2: GloVe / Word2Vec 平均文档向量 + LR
<code>task3_bert.py</code>	Task 3: BERT 微调
<code>requirements.txt</code>	依赖清单
<code>README.md</code>	数据准备与运行说明

A.2 数据与模型准备

体积较大的数据与预训练文件统一放在代码目录的 `HW-1/` 下（不纳入版本管理）：

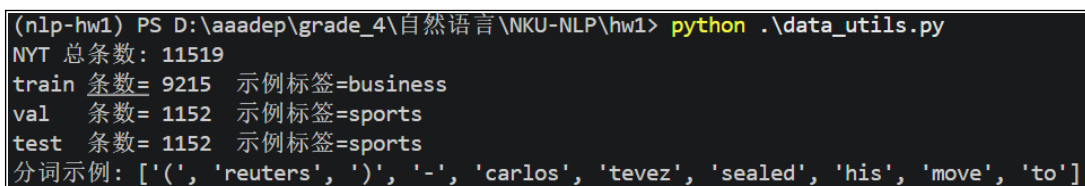
- `HW-1/nyt.csv`、`HW-1/ag.csv`；
- 从 `glove.6B.zip` 中解压出的 `glove.6B.100d.txt`（仅需 100 维这一份）；

- BERT 模型目录 HW-1/bert-base-uncased/; 若该目录不存在, 程序会自动从 Hugging Face 下载 google-bert/bert-base-uncased。

A.3 运行方式

配置好环境并准备好数据后, 依次运行三个脚本即可复现全部结果:

```
1 conda activate nlp-hw1
2 python data_utils.py          # optional: check the split sizes
3 python task1_bow.py           # Task 1: Bag-of-Words
4 python task2_word2vec.py      # Task 2: GloVe / Word2Vec
5 python task3_bert.py          # Task 3: BERT fine-tuning
```



```
(nlp-hw1) PS D:\aaadep\grade_4\自然语言\NKU-NLP\hw1> python .\data_utils.py
NYT 总条数: 11519
train 条数= 9215 示例标签=business
val 条数= 1152 示例标签=sports
test 条数= 1152 示例标签=sports
分词示例: ['(', 'reuters', ')', '-', 'carlos', 'tevez', 'sealed', 'his', 'move', 'to']
```

图 A.8: 数据读取与划分自检结果: 终端打印 NYT 总条数、train/val/test 划分条数与分词示例